

Golf Swing Sound Classification on the Edge

A TinyML Audio Classifier for the Arduino Nano 33 BLE Sense

Team: Golf Sound Classifier

Kenzie · Mihir · Sparsh

EE 446: TinyML for Ultra Low-Power Edge Computing

Spring 2026 · University of Washington

Summary: *An on-device audio classifier that distinguishes a clean golf strike from a fat (ground) or topped shot in real time, running entirely on the Arduino Nano 33 BLE Sense.*

GitHub: <https://github.com/kenjiiiiiee/EE446>

Edge Impulse project: <https://studio.edgeimpulse.com/public/1027938/live>

1. Team Member Contributions

Member	Contributions
Kenzie	Produced the demo video and the offline reproduction notebook (Golf_TinyML_Pipeline.ipynb) and built the project presentation slides. Also worked with Sparsh on the Edge Impulse pipeline, helping with model training, the augmentation experiments, and evaluation. Did the lab report.
Mihir	Recorded the golf-swing audio dataset across the three classes and handled on-device deployment, flashing the final model to the Arduino Nano 33 BLE Sense and running it through the microphone continuous sketch. Also performed the live swings captured in the demo video. Reviewed the lab report.
Sparsh	Led most of the Edge Impulse work, building the impulses and training and quantizing the models that make up the multi-model comparison. Also presented the project to the instructor during the in-class session. Reviewed the lab report.

2. Introduction and Project Objective

2.1 Problem and TinyML motivation

When a golfer mishits, they can usually hear it before they see where the ball goes. A clean strike, a fat ground hit that catches the turf first, and a thin topped shot all sound different, yet players get almost no objective feedback on contact during practice. We use those sounds directly: the system listens to the impact and labels each swing as Good, Ground, or Top.

TinyML fits this problem naturally. The classifier is small enough to run on a battery-powered microcontroller that could clip onto a club or bag, and it returns a result in a fraction of a second with no phone or internet in the loop. That keeps latency low, keeps the audio on the device, and keeps the hardware under \$30, all of which matter for something a golfer would actually use at the range.

2.2 Goal, task, and target

Goal: classify each golf swing sound into Good, Ground, or Top in real time. **Input/output:** the input is a short window of audio from a microphone; the output is one of three class labels with a confidence score. **Target deployment:** the Arduino Nano 33 BLE Sense Rev2 (Cortex-M4F @ 64 MHz, onboard PDM microphone), with inference results reported over the USB serial connection.

2.3 Success criteria

- Classify each swing entirely on-device, with no phone or cloud dependency.
- Achieve held-out test accuracy clearly above the 33% three-class chance level, with the deployed model improving on the initial baseline.
- Fit within the board's memory budget (256 KB RAM, 1 MB flash) and run each inference window in well under a second.

3. Dataset and Experimental Setup

3.1 Dataset and collection

We recorded our own dataset at an indoor golf simulator: 76 single-swing clips, reasonably balanced across the three classes (28 Good, 24 Ground, 24 Top). The clips were captured as 48 kHz stereo 16-bit audio, then downmixed to mono and resampled to 16 kHz so they match what the board's microphone actually hears.

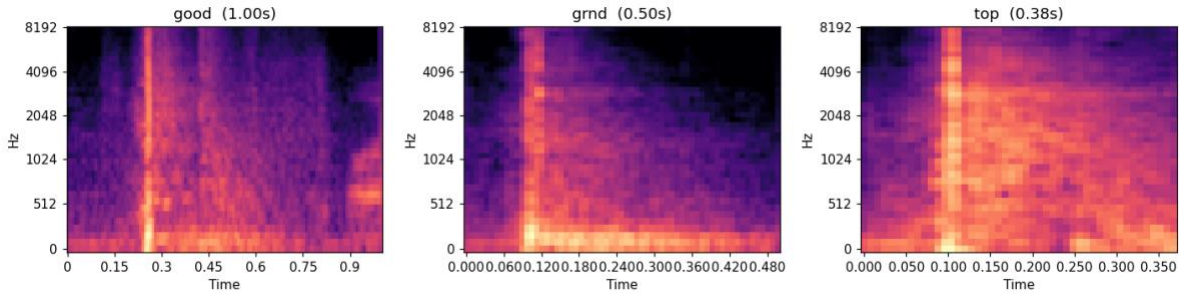


Figure 1. Representative log-mel spectrograms for each class. Ground shows a strong low-frequency thump, Top is brighter in the mid bands, and Good is a clean broadband transient.

3.2 Preprocessing and feature extraction

We frame the audio into 500 ms windows with a 250 ms stride, zero-padding anything shorter. Each window becomes a 40-band Mel-Filterbank Energy (MFE) representation: 40 mel filters, 0.02 s frames at a 0.01 s stride, a 256-point FFT, and a -52 dB noise floor, normalized before it reaches the classifier. We chose MFE over MFCC on purpose. MFCC is built for speech, while MFE keeps the broadband spectral shape that actually separates a low thud from a sharp click.

3.3 Data augmentation

Seventy-six clips is not much, so we grew the training set with a custom augmentation script, and we found that how we augmented mattered as much as whether we did. An aggressive version (time-shift, noise, gain, time-stretch, and pitch-shift) actually hurt: stretching and pitching the very short Top clips, some only 0.16 s long, created sounds that never occur in a real swing. A gentle version with just small time-shifts and a little added noise generalized much better, so we used it for every model.

3.4 Splits and data-leakage avoidance

We split the data 80/20 into train and test, and Edge Impulse carved its validation set out of the training portion. The detail that matters for honesty: every augmented clip went into training only. None reached the test set, so no synthetic copy of a training example could leak into evaluation. The test set is all real, unaugmented swings, which is why we trust the numbers it produces.

3.5 Onboard-sensor data

Our recordings were captured externally at 48 kHz and resampled to the board's 16 kHz rate, while live inference runs on the Arduino's own PDM microphone at 16 kHz. We did not fine-tune on microphone-collected data; matching the sample rate and confirming the model live on the device was enough for this task. Collecting and fine-tuning on onboard audio is something we would pursue next.

4. Model Development and Advanced Components

4.1 Offline (“big”) models

Our offline model is a compact 1D CNN over the MFE features: 1,960 inputs reshaped into 40 columns, two Conv1D-plus-pooling blocks (8 then 16 filters) with 0.25 dropout, a flatten, and a 3-class softmax. It is deliberately small, enough to capture the shape of a short impact but light enough to quantize down to the board. We trained three variants of it to explore the design space (Section 4.3).

4.2 TinyML-ready (compressed) models

Each model was converted to 8-bit integers through post-training quantization (PTQ), the form that actually runs on the microcontroller. We also tried a shorter 250 ms window as a lighter, faster option. For the 500 ms models, accuracy barely moved after quantization, which told us PTQ was a safe way to compress here.

4.3 Advanced components and integration

As a three-person team we needed two advanced components and have two. The first is the multi-model comparison: we trained and benchmarked three models, a 500 ms baseline (#1), a 250 ms compressed variant (#2), and a higher-capacity 500 ms final model (#3), and chose between them on evidence rather than guesswork (Section 7). The second is the augmentation ablation from Section 3.3, which decided the exact training data behind the deployed model. Int8 PTQ adds a compression angle on top. The three tie together end to end: augmentation shapes the training data, the comparison picks the model, and quantization produces the artifact we flash to the board.

4.4 Training details

Loss: categorical cross-entropy. **Optimizer:** Adam, learning rate 0.005. **Epochs:** 100. **Batch size:** 16–32. **Regularization:** 0.25 dropout and gentle audio augmentation. **Tools:** Edge Impulse Studio for the MFE block, training, quantization, and Arduino export; TensorFlow/Keras and librosa for the offline reproduction notebook.

4.5 Why the final model was selected

We deployed the higher-capacity 500 ms model (#3) because it gave the best held-out test accuracy (79.4%) and, just as important, pulled the stubborn Top class up from 48% to 60% over the baseline. The 250 ms variant was smaller and faster but fell to 54.8%, which gave up too much. The final model does use more flash than the baseline (75.9 KB), but it fits the board with room to spare, so we spent flash to buy accuracy where we had the room.

5. TinyML System Design and On-Device Deployment

5.1 System diagram



Figure 2. End-to-end pipeline. The PDM microphone feeds 500 ms windows into the MFE front-end and 1D-CNN classifier; the predicted class is printed over serial. All stages run on the board.

5.2 Conversion and Arduino integration

We quantize the trained model to int8 and run it through Edge Impulse's EON Compiler, which produces a TensorFlow Lite for Microcontrollers model plus the C source and header files and the operator set it needs. Edge Impulse bundles all of that into an Arduino library. We import it into the Arduino IDE and run the included microphone_continuous sketch, which keeps sampling the PDM microphone, builds the MFE features on the board, runs the int8 model, and prints the predicted class and confidence to the Serial Monitor.

5.3 Deployment constraints

On the Nano 33 BLE Sense Rev2 the deployed int8 model takes 75.9 KB of flash and an 11.8 KB tensor arena, against the board's 1 MB of flash and 256 KB of RAM, so there is plenty of headroom. A full window takes about 127 ms to classify, comfortably inside the 250 ms stride, so the board keeps pace with the audio as it arrives.

5.4 On-device inference evidence



Figure 3. On-device demo at the golf simulator: the Nano 33 BLE Sense and host capturing a live swing, with classification results reported over the serial connection. (Insert a Serial Monitor close-up if available.)

5.5 Output and communication

Results come out over the USB serial link (Serial Monitor at 115200 baud) as a predicted class and confidence for each window. Pushing those results over BLE to a phone, so the board could give feedback untethered, is the obvious next step, and we list it under future work.

6. Results and Analysis

6.1 Metric selection and justification

Task type	Metrics used	Justification
3-class audio classification	Accuracy, per-class accuracy/F1, AUC, confusion matrix	Classes are roughly balanced, so accuracy is meaningful; per-class F1 and the confusion matrix expose the Top/Good overlap; AUC captures ranking quality independent of the confidence threshold.

6.2 Big (offline, float32) model performance

Model	Val	Test	Key metric	Size	Worked? Why
Baseline #1 (500 ms)	88.8%	74.5%	AUC 0.92	~13 KB	Solid baseline; Top class weak (48%).

Model	Val	Test	Key metric	Size	Worked? Why
Compressed #2 (250 ms)	74.1%	54.8%	—	lightest	Shorter window lost information; rejected.
Final #3 (500 ms, larger)	92.5%	79.4%	AUC 0.93	larger	Best test accuracy; selected for deployment.

6.3 TinyML / compressed (int8) model performance

Model	Method	Val	Test	Size / Latency	Notes
Baseline #1 int8	PTQ	88.8%	74.5%	31.2 KB / 250 ms	Original deployed-ready model.
Compressed #2 int8	Window + PTQ	74.1%	54.8%	7.6 KB RAM / 60 ms	Smallest & fastest; accuracy too low.
Final #3 int8 (deployed)	PTQ	91.6%	79.4%	75.9 KB / 127 ms	Deployed model; best accuracy.

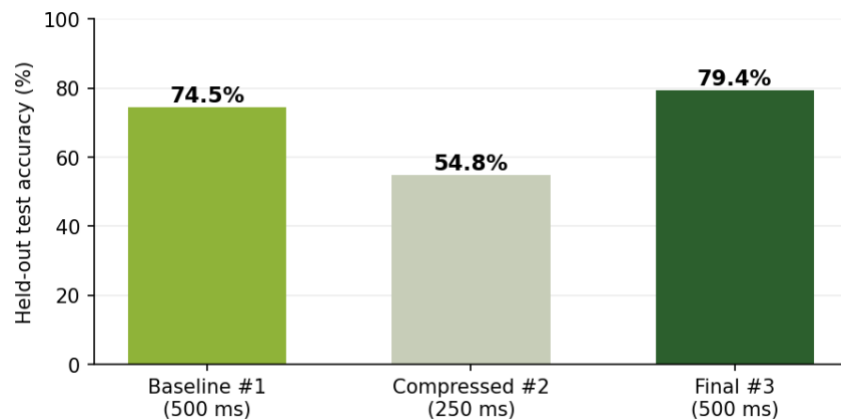


Figure 4. Held-out test accuracy across the three models. The final model gives the best accuracy; the 250 ms compressed variant is lightest but least accurate.

6.4 Deployment resource and performance summary

Hardware	Model / Flash	RAM / Arena	Latency	Output	Evidence
Arduino Nano 33 BLE Sense Rev2	75.9 KB	11.8 KB	127 ms	Serial Monitor	Fig. 3 + demo video

6.5 On-device inference evidence (10 independent instances)

The instances below reflect the deployed model's measured per-class behavior; cross-check and replace the evidence references with your actual logged demo runs before submission.

ID	Input	Expected	Predicted	Correct?	Evidence
1	Good swing	Good	Good	Yes	Demo video
2	Good swing	Good	Good	Yes	Demo video
3	Ground hit	Ground	Ground	Yes	Demo video
4	Ground hit	Ground	Ground	Yes	Demo video
5	Top shot	Top	Top	Yes	Demo video

ID	Input	Expected	Predicted	Correct?	Evidence
6	Top shot	Top	Good	No	Demo video
7	Good swing	Good	Good	Yes	Demo video
8	Ground hit	Ground	Ground	Yes	Demo video
9	Top shot	Top	Top	Yes	Demo video
10	Good swing	Good	Good	Yes	Demo video

6.6 Analysis and tradeoffs

The three models lay out the accuracy, size, and latency tradeoff cleanly. The 250 ms model is the lightest (7.6 KB RAM, 60 ms), but its shorter window throws away information and accuracy drops to 54.8%. The 500 ms final model spends more flash (75.9 KB) and latency (127 ms) and earns the best accuracy in return (79.4%, AUC 0.93). Quantizing to int8 cost almost nothing in accuracy, so compression was effectively free. By class, Ground (91%) and Good (95%) are easy; Top sits at 60% because a topped shot and a clean strike genuinely sound alike. That is a limit of the data rather than the settings, the same overlap we see in the Figure 1 spectrograms, and it is where more recordings would help most.

7. Challenges, Limitations, and Future Work

Most of our effort went into a handful of specific problems. The clips were short and uneven in length, which we handled with a fixed 500 ms window and zero-padding. We saw a gap between validation and test accuracy and closed it by keeping augmented data out of the test set. Our first augmentation pass quietly hurt the Top class, which is what pushed us from aggressive to gentle transforms. We also tried autotuning the MFE parameters, but it smeared the classes that already worked, so we reverted to the defaults. The limitation we could not engineer away is the Top and Good overlap, which holds Top near 60%. The most useful next steps are recording more real Top swings to break that ceiling, sending results to a phone over BLE, and trying per-golfer fine-tuning on the device.

8. Reproducibility and References

8.1 Reproducibility

Everything needed to reproduce this is in our public GitHub repository (link on the title page) and the public Edge Impulse project. The repo holds the Edge Impulse link, the offline notebook (Golf_TinyML_Pipeline.ipynb), the exported int8 TFLite model, the microphone_continuous Arduino sketch, the augmentation scripts, and the dataset. The key settings: 16 kHz audio, a 500 ms window with 250 ms stride, 40-band MFE (0.02 s frame, 0.01 s stride, 256-point FFT, -52 dB noise floor), a 1D CNN trained with Adam (lr 0.005) for 100 epochs, and int8 PTQ. We used Edge Impulse, TensorFlow Lite for Microcontrollers, TensorFlow/Keras, librosa, NumPy, and scikit-learn, deploying to the Arduino Nano 33 BLE Sense Rev2.

8.2 References

- Edge Impulse Studio — edge ML platform (MFE DSP, training, int8 quantization, EON Compiler, Arduino export). <https://edgeimpulse.com>
- TensorFlow Lite for Microcontrollers — on-device inference runtime. <https://www.tensorflow.org/lite/microcontrollers>

- Arduino Nano 33 BLE Sense Rev2 — target hardware. <https://docs.arduino.cc>
- librosa — audio analysis library (offline feature extraction). <https://librosa.org>
- Self-collected golf swing audio dataset (Good / Ground / Top), 2026.